Computer Vision---Tracking

桑农 王岳环

Slides from CV course with Prof. Trevor Darrel

Tracking scenarios

Why tracking?

Follow a point

Follow a template

Follow a changing template

Follow all the elements of a moving person, fit a model to it.

Tracking by detection vs. tracking by matching

Things to consider in tracking

What are the dynamics of the thing being tracked?

How is it observed?

Three main issues in tracking

- **Prediction:** we have seen $\boldsymbol{y}_0, \ldots, \boldsymbol{y}_{i-1}$ — what state does this set of measurements predict for the i'th frame? to solve this problem, we need to obtain a representation of $P(\boldsymbol{X}_i | \boldsymbol{Y}_0 = \boldsymbol{y}_0, \ldots, \boldsymbol{Y}_{i-1} = \boldsymbol{y}_{i-1})$.
- **Data association:** Some of the measurements obtained from the i-th frame may tell us about the object's state. Typically, we use $P(\boldsymbol{X}_i | \boldsymbol{Y}_0 = \boldsymbol{y}_0, \ldots, \boldsymbol{Y}_{i-1} = \boldsymbol{y}_{i-1})$ to identify these measurements.
- **Correction:** now that we have $\boldsymbol{y}_i$ — the relevant measurements — we need to compute a representation of $P(\boldsymbol{X}_i | \boldsymbol{Y}_0 = \boldsymbol{y}_0, \ldots, \boldsymbol{Y}_i = \boldsymbol{y}_i)$.

Simplifying Assumptions

- **Only the immediate past matters:** formally, we require

$$P(\boldsymbol{X}_i|\boldsymbol{X}_1,\ldots,\boldsymbol{X}_{i-1}) = P(\boldsymbol{X}_i|\boldsymbol{X}_{i-1})$$

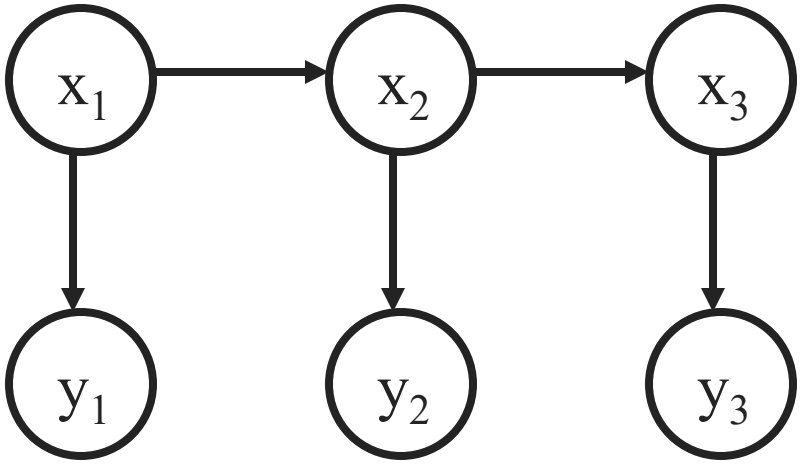
  This assumption hugely simplifies the design of algorithms, as we shall see; furthermore, it isn't terribly restrictive if we're clever about interpreting $\boldsymbol{X}_i$ as we shall show in the next section.

- **Measurements depend only on the current state:** we assume that $\boldsymbol{Y}_i$ is conditionally independent of all other measurements given $\boldsymbol{X}_i$. This means that

$$P(\boldsymbol{Y}_i,\boldsymbol{Y}_j,\ldots\boldsymbol{Y}_k|\boldsymbol{X}_i) = P(\boldsymbol{Y}_i|\boldsymbol{X}_i)P(\boldsymbol{Y}_j,\ldots,\boldsymbol{Y}_k|\boldsymbol{X}_i)$$

  Again, this isn't a particularly restrictive or controversial assumption, but it yields important simplifications.

Kalman filter graphical model and corresponding factorized joint probability



$$P(x_1, x_2, x_3, y_1, y_2, y_3) =$$
$$P(x_1)P(y_1|x_1)P(x_2|x_1)P(y_2|x_2)P(x_3|x_2)P(y_3|x_3)$$

Tracking as induction

Make a measurement starting in the 0th frame

Then: assume you have an estimate at the ith frame, after the measurement step.

Show that you can do prediction for the i+1th frame, and measurement for the i+1th frame.

Base case

Firstly, we assume that we have $P(\boldsymbol{X}_0)$

$$P(\boldsymbol{X}_0|\boldsymbol{Y}_0 = \boldsymbol{y}_0) = \frac{P(\boldsymbol{y}_0|\boldsymbol{X}_0)P(\boldsymbol{X}_0)}{P(\boldsymbol{y}_0)}$$

$$\propto P(\boldsymbol{y}_0|\boldsymbol{X}_0)P(\boldsymbol{X}_0)$$

Prediction step

Prediction

Prediction involves representing

$$P(\boldsymbol{X}_i|\boldsymbol{y}_0,\ldots,\boldsymbol{y}_{i-1})$$

given

$$P(\boldsymbol{X}_{i-1}|\boldsymbol{y}_0,\ldots,\boldsymbol{y}_{i-1}).$$

Our independence assumptions make it possible to write

$$\begin{aligned} P(\boldsymbol{X}_i|\boldsymbol{y}_0,\ldots,\boldsymbol{y}_{i-1}) &= \int P(\boldsymbol{X}_i,\boldsymbol{X}_{i-1}|\boldsymbol{y}_0,\ldots,\boldsymbol{y}_{i-1})d\boldsymbol{X}_{i-1} \\ &= \int P(\boldsymbol{X}_i|\boldsymbol{X}_{i-1},\boldsymbol{y}_0,\ldots,\boldsymbol{y}_{i-1})P(\boldsymbol{X}_{i-1}|\boldsymbol{y}_0,\ldots,\boldsymbol{y}_{i-1})d\boldsymbol{X}_{i-1} \\ &= \int P(\boldsymbol{X}_i|\boldsymbol{X}_{i-1})P(\boldsymbol{X}_{i-1}|\boldsymbol{y}_0,\ldots,\boldsymbol{y}_{i-1})d\boldsymbol{X}_{i-1} \end{aligned}$$

Update step

Correction

Correction involves obtaining a representation of

$$P(\boldsymbol{X}_i|\boldsymbol{y}_0,\ldots,\boldsymbol{y}_i)$$

given

$$P(\boldsymbol{X}_i|\boldsymbol{y}_0,\ldots,\boldsymbol{y}_{i-1})$$

$$P(y_0,\ldots,y_i) = \int P(y_i \mid \boldsymbol{X}_i, y_0 \ldots y_{i-1})\, P(\boldsymbol{X}_i, y_0 \ldots y_{i-1}) d\boldsymbol{X}_i$$
$$= \int P(y_i \mid \boldsymbol{X}_i, y_0 \ldots y_{i-1})\, P(\boldsymbol{X}_i | y_0 \ldots y_{i-1}) P(y_0 \ldots y_{i-1}) d\boldsymbol{X}_i$$
$$= P(y_0 \ldots y_{i-1}) \int P(y_i \mid \boldsymbol{X}_i)\, P(\boldsymbol{X}_i | y_0 \ldots y_{i-1}) d\boldsymbol{X}_i$$

Our independence assumptions make it possible to write

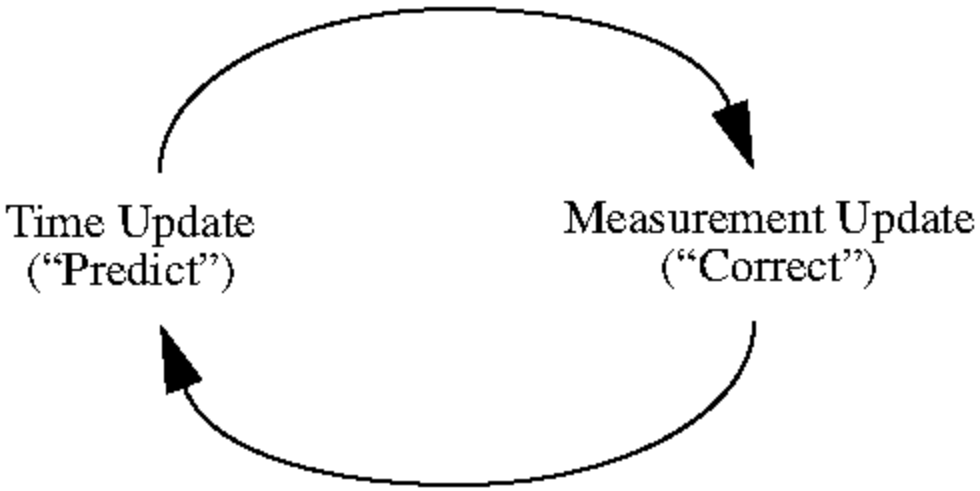
$$\begin{aligned}
P(\boldsymbol{X}_i|\boldsymbol{y}_0,\ldots,\boldsymbol{y}_i) &= \frac{P(\boldsymbol{X}_i,\boldsymbol{y}_0,\ldots,\boldsymbol{y}_i)}{P(\boldsymbol{y}_0,\ldots,\boldsymbol{y}_i)} \\
&= \frac{P(\boldsymbol{y}_i|\boldsymbol{X}_i,\boldsymbol{y}_0,\ldots,\boldsymbol{y}_{i-1})P(\boldsymbol{X}_i|\boldsymbol{y}_0,\ldots,\boldsymbol{y}_{i-1})P(\boldsymbol{y}_0,\ldots,\boldsymbol{y}_{i-1})}{P(\boldsymbol{y}_0,\ldots,\boldsymbol{y}_i)} \\
&= P(\boldsymbol{y}_i|\boldsymbol{X}_i)P(\boldsymbol{X}_i|\boldsymbol{y}_0,\ldots,\boldsymbol{y}_{i-1})\frac{P(\boldsymbol{y}_0,\ldots,\boldsymbol{y}_{i-1})}{P(\boldsymbol{y}_0,\ldots,\boldsymbol{y}_i)} \\
&= \frac{P(\boldsymbol{y}_i|\boldsymbol{X}_i)P(\boldsymbol{X}_i|\boldsymbol{y}_0,\ldots,\boldsymbol{y}_{i-1})}{\int P(\boldsymbol{y}_i|\boldsymbol{X}_i)P(\boldsymbol{X}_i|\boldsymbol{y}_0,\ldots,\boldsymbol{y}_{i-1})d\boldsymbol{X}_i}
\end{aligned}$$

The Kalman Filter

Key ideas:

- Linear models interact uniquely well with Gaussian noise - make the prior Gaussian, everything else Gaussian and the calculations are easy
- Gaussians are really easy to represent --- once you know the mean and covariance, you're done

The Kalman Filter



[figure from http://www.cs.unc.edu/~welch/kalman/kalmanIntro.html]

The Kalman Filter in 1D

Dynamic Model

$$x_i \sim N(d_i x_{i-1}, \sigma_{d_i}^2)$$

$$y_i \sim N(m_i x_i, \sigma_{m_i}^2)$$

Notation

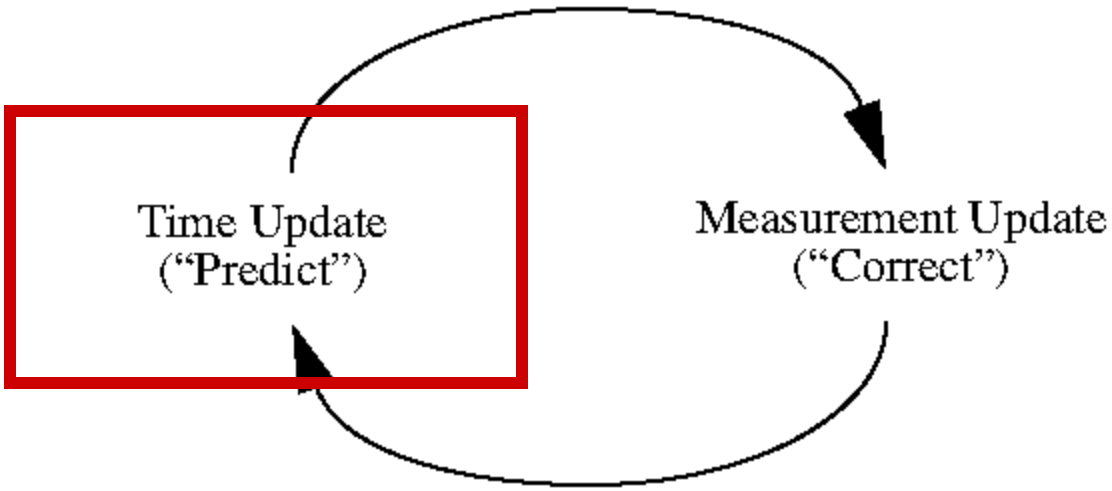
mean of $P(X_i|y_0, \ldots, y_{i-1})$ as $\overline{X}_i^-$ ← Predicted mean

mean of $P(X_i|y_0, \ldots, y_i)$ as $\overline{X}_i^+$ ← Corrected mean

the standard deviation of $P(X_i|y_0, \ldots, y_{i-1})$ as σ_i^-

of $P(X_i|y_0, \ldots, y_i)$ as σ_i^+

The Kalman Filter



Prediction for 1D Kalman filter

The new state is obtained by

multiplying old state by known constant

adding zero-mean noise

$$x_i \sim N(d_i x_{i-1}, \sigma_{d_i}^2)$$

Therefore, predicted mean for new state is

constant times mean for old state

Old variance is normal random variable

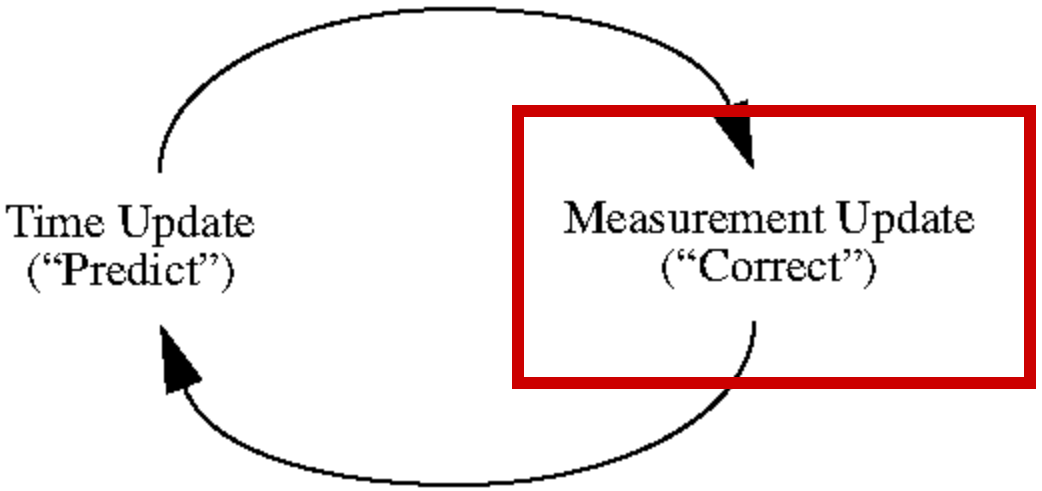
variance is multiplied by square of constant

and variance of noise is added.

$$\overline{X}_i^- = d_i \overline{X}_{i-1}^+$$

$$(\sigma_i^-)^2 = \sigma_{d_i}^2 + (d_i \sigma_{i-1}^+)^2$$

The Kalman Filter



Measurement update for 1D Kalman filter

$$x_i^+ = \left(\frac{\overline{x_i^-}\,\sigma_{m_i}^2 + m_i y_i (\sigma_i^-)^2}{\sigma_{m_i}^2 + m_i^2 (\sigma_i^-)^2} \right)$$

$$\sigma_i^+ = \sqrt{\left(\frac{\sigma_{m_i}^2 (\sigma_i^-)^2}{(\sigma_{m_i}^2 + m_i^2 (\sigma_i^-)^2)} \right)}$$

Notice:

if measurement noise is small,

we rely mainly on the measurement,

if it's large, mainly on the prediction

σ does not depend on y

Dynamic Model:

$$x_i \sim N(d_i x_{i-1}, \sigma_{d_i})$$

$$y_i \sim N(m_i x_i, \sigma_{m_i})$$

Start Assumptions: $\overline{x_0^-}$ and σ_0^- are known

Update Equations: Prediction

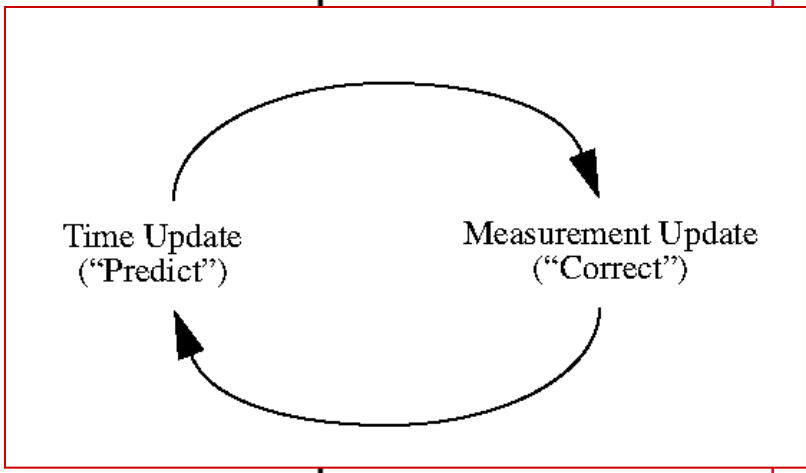
$$\overline{x_i^-} = d_i \overline{x}_{i-1}^+$$

$$\sigma_i^- = \sqrt{\sigma_{d_i}^2 + (d_i \sigma_{i-1}^+)^2}$$

Update Equations: Correction

$$x_i^+ = \left(\frac{\overline{x_i^-}\,\sigma_{m_i}^2 + m_i y_i (\sigma_i^-)^2}{\sigma_{m_i}^2 + m_i^2 (\sigma_i^-)^2} \right)$$

$$\sigma_i^+ = \sqrt{\left(\frac{\sigma_{m_i}^2 (\sigma_i^-)^2}{(\sigma_{m_i}^2 + m_i^2 (\sigma_i^-)^2)} \right)}$$



Kalman filter for computing an on-line average

What Kalman filter parameters and initial conditions should we pick so that the optimal estimate for x at each iteration is just the average of all the observations seen so far?

Dynamic Model:

$$x_i \sim N(d_i x_{i-1}, \sigma_{d_i})$$

$$y_i \sim N(m_i x_i, \sigma_{m_i})$$

Start Assumptions: $\bar{x}_0^-$ and σ_0^- are known

Update Equations: Prediction

$$\bar{x}_i^- = d_i \bar{x}_{i-1}^+$$

$$\sigma_i^- = \sqrt{\sigma_{d_i}^2 + (d_i \sigma_{i-1}^+)^2}$$

Update Equations: Correction

$$x_i^+ = \left(\frac{\bar{x}_i^- \sigma_{m_i}^2 + m_i y_i (\sigma_i^-)^2}{\sigma_{m_i}^2 + m_i^2 (\sigma_i^-)^2} \right)$$

$$\sigma_i^+ = \sqrt{\left(\frac{\sigma_{m_i}^2 (\sigma_i^-)^2}{(\sigma_{m_i}^2 + m_i^2 (\sigma_i^-)^2)} \right)}$$

Kalman filter model

$$d_i = 1, m_i = 1, \sigma_{d_i} = 0, \sigma_{m_i} = 1$$

Initial conditions

$$\bar{x}_0^- = 0 \quad \sigma_0^- = \infty$$

Iteration	0	1	2
$\bar{x}_i^-$	0	y_0	$\frac{y_0 + y_1}{2}$
$\bar{x}_i^+$	y_0	$\frac{y_0 + y_1}{2}$	$\frac{y_0 + y_1 + y_2}{3}$
σ_i^-	∞	1	$\frac{1}{\sqrt{2}}$
σ_i^+	1	$\frac{1}{\sqrt{2}}$	$\frac{1}{\sqrt{3}}$

What happens if the x dynamics are given a non-zero variance?

Kalman filter model

Dynamic Model:

$$x_i \sim N(d_i x_{i-1}, \sigma_{d_i})$$

$$y_i \sim N(m_i x_i, \sigma_{m_i})$$

Start Assumptions: $\bar{x}_0^-$ and σ_0^- are known

Update Equations: Prediction

$$\bar{x}_i^- = d_i \bar{x}_{i-1}^+$$

$$\sigma_i^- = \sqrt{\sigma_{d_i}^2 + (d_i \sigma_{i-1}^+)^2}$$

Update Equations: Correction

$$x_i^+ = \left(\frac{\bar{x}_i^- \sigma_{m_i}^2 + m_i y_i (\sigma_i^-)^2}{\sigma_{m_i}^2 + m_i^2 (\sigma_i^-)^2} \right)$$

$$\sigma_i^+ = \sqrt{\left(\frac{\sigma_{m_i}^2 (\sigma_i^-)^2}{(\sigma_{m_i}^2 + m_i^2 (\sigma_i^-)^2)} \right)}$$

$$d_i = 1, m_i = 1, \sigma_{d_i} = 1, \sigma_{m_i} = 1$$

Initial conditions

$$\bar{x}_0^- = 0 \quad \sigma_0^- = \infty$$

Iteration	0	1	2
$\bar{x}_i^-$	0	y_0	$\frac{y_0 + 2y_1}{3}$
$\bar{x}_i^+$	y_0	$\frac{y_0 + 2y_1}{3}$	$\frac{y_0 + 2y_1 + 5y_2}{8}$
σ_i^-	∞	$\sqrt{2}$	$\sqrt{\frac{5}{3}}$
σ_i^+	1	$\sqrt{\frac{2}{3}}$	$\sqrt{\frac{5}{8}}$

Linear dynamic models

A linear dynamic model has the form

$$\mathbf{x}_i = N\left(\mathbf{D}_{i-1}\mathbf{x}_{i-1}; \Sigma_{d_i}\right)$$

$$\mathbf{y}_i = N\left(\mathbf{M}_i\mathbf{x}_i; \Sigma_{m_i}\right)$$

This is much, much more general than it looks, and extremely powerful

Examples of linear state space models

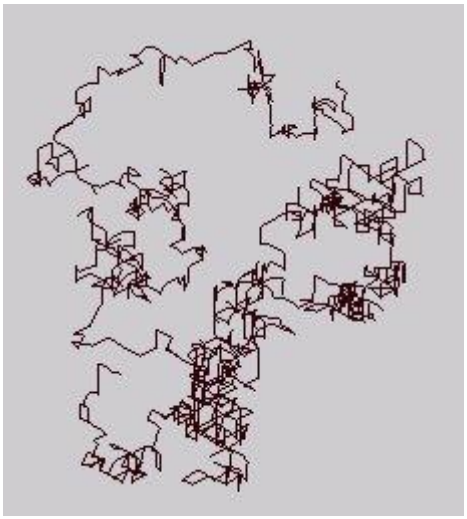
Drifting points

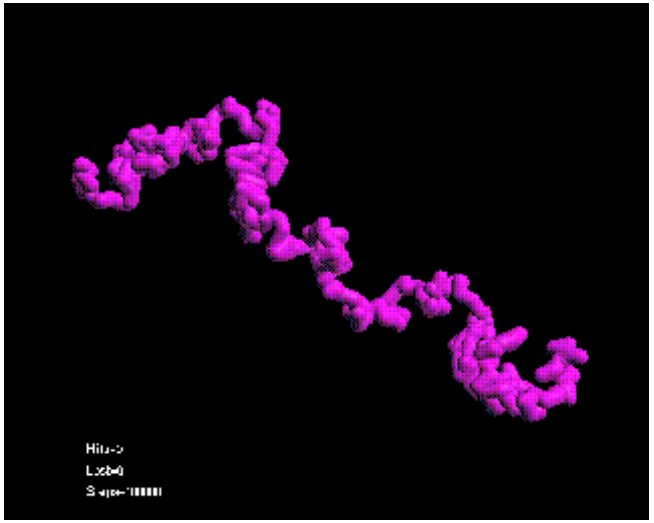
assume that the new position of the point is the old one, plus noise

D = Identity

$$\mathbf{x}_i = N\left(\mathbf{D}_{i-1}\mathbf{x}_{i-1}; \Sigma_{d_i}\right)$$

$$\mathbf{y}_i = N\left(\mathbf{M}_i\mathbf{x}_i; \Sigma_{m_i}\right)$$





cic.nist.gov/lipman/sciviz/images/random3.gif
http://www.grunch.net/synergetics/images/random3.jpg

Constant velocity

$$\mathbf{x}_i = N\left(\mathbf{D}_{i-1}\mathbf{x}_{i-1}; \Sigma_{d_i}\right)$$

We have

$$\mathbf{y}_i = N\left(\mathbf{M}_i\mathbf{x}_i; \Sigma_{m_i}\right)$$

$$u_i = u_{i-1} + \Delta t v_{i-1} + \varepsilon_i$$

$$v_i = v_{i-1} + \varsigma_i$$

(the Greek letters denote noise terms)

Stack (u, v) into a single state vector

which is the form we had above $\begin{pmatrix} u \\ v \end{pmatrix}_i = \begin{pmatrix} 1 & \Delta t \\ 0 & 1 \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix}_{i-1} + \text{noise}$

$\mathbf{x_i}$ $\mathbf{D_{i-1}}$ $\mathbf{x_{i-1}}$

Constant acceleration

$$\mathbf{x}_i = N\left(\mathbf{D}_{i-1}\mathbf{x}_{i-1}; \Sigma_{d_i}\right)$$

$$\mathbf{y}_i = N\left(\mathbf{M}_i\mathbf{x}_i; \Sigma_{m_i}\right)$$

We have

$$u_i = u_{i-1} + \Delta t v_{i-1} + \varepsilon_i$$
$$v_i = v_{i-1} + \Delta t a_{i-1} + \varsigma_i$$
$$a_i = a_{i-1} + \xi_i$$

(the Greek letters denote noise terms)

Stack (u, v) into a single state vector

which is the form we had above

$$\begin{pmatrix} u \\ v \\ a \end{pmatrix}_i = \begin{pmatrix} 1 & \Delta t & 0 \\ 0 & 1 & \Delta t \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} u \\ v \\ a \end{pmatrix}_{i-1} + \text{noise}$$

$\mathbf{x_i}$ $\mathbf{D_{i-1}}$ $\mathbf{x_{i-1}}$

Periodic motion

$$\mathbf{x}_i = N\left(\mathbf{D}_{i-1}\mathbf{x}_{i-1}; \Sigma_{d_i}\right)$$

$$\mathbf{y}_i = N\left(\mathbf{M}_i\mathbf{x}_i; \Sigma_{m_i}\right)$$

Assume we have a point, moving on a line with a periodic movement defined with a differential eq:

can be defined as $$\frac{d^2p}{dt^2} = -p$$

with state defined as stacked position and velocity u=(p, v)

$$\frac{d\boldsymbol{u}}{dt} = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} \boldsymbol{u} = \mathcal{S}\boldsymbol{u}$$

Periodic motion

$$\mathbf{x}_i = N\left(\mathbf{D}_{i-1}\mathbf{x}_{i-1}; \Sigma_{d_i}\right)$$

$$\mathbf{y}_i = N\left(\mathbf{M}_i\mathbf{x}_i; \Sigma_{m_i}\right)$$

$$\frac{d\boldsymbol{u}}{dt} = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} \boldsymbol{u} = \mathcal{S}\boldsymbol{u}$$

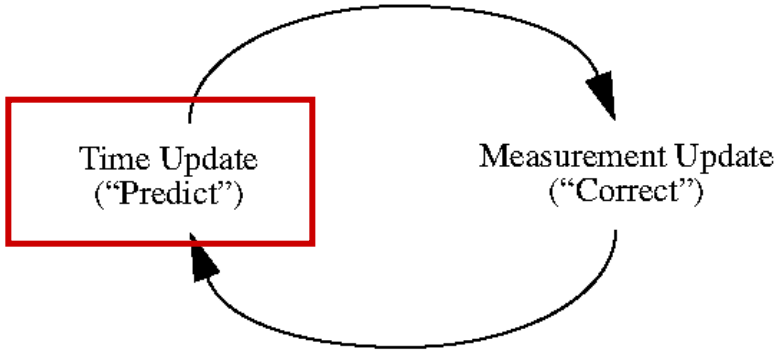
Take discrete approximation….(e.g., forward Euler integration with Δt stepsize.)

$$\begin{aligned} \boldsymbol{u}_i &= \boldsymbol{u}_{i-1} + \Delta t \frac{d\boldsymbol{u}}{dt} \\ &= \boldsymbol{u}_{i-1} + \Delta t \mathcal{S}\boldsymbol{u}_{i-1} \\ &= \begin{pmatrix} 1 & \Delta t \\ -\Delta t & 1 \end{pmatrix} \boldsymbol{u}_{i-1} \end{aligned}$$

$\mathbf{x}_i$ $\mathbf{D}_{i-1}$ $\mathbf{x}_{i-1}$

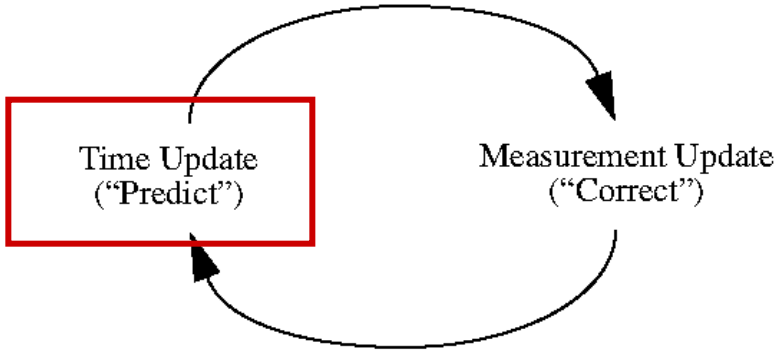
n-D

Generalization to n-D is straightforward but more complex.



n-D Prediction

Generalization to n-D is straightforward but more complex.



Prediction:

Multiply estimate at prior time with forward model:

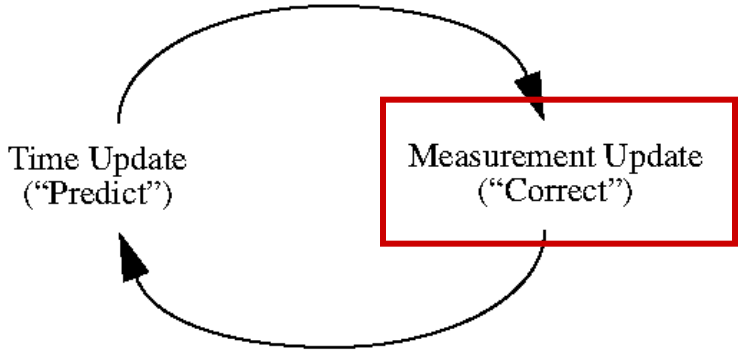
$$\overline{\boldsymbol{x}}_i^- = \mathcal{D}_i \overline{\boldsymbol{x}}_{i-1}^+$$

Propagate covariance through model and add new noise:

$$\Sigma_i^- = \Sigma_{d_i} + \mathcal{D}_i \sigma_{i-1}^+ \mathcal{D}_i$$

n-D Correction

Generalization to n-D is straightforward but more complex.



Correction:

Update *a priori* estimate with measurement to form *a posteriori*

n-D correction

Find linear filter on innovations

$$\overline{\boldsymbol{x}}_i^+ = \overline{\boldsymbol{x}}_i^- + \mathcal{K}_i \left[\boldsymbol{y}_i - \mathcal{M}_i \overline{\boldsymbol{x}}_i^- \right]$$

which minimizes *a posteriori* error covariance:

$$E\left[\left(x - \overline{x^+}\right)^T \left(x - \overline{x^+}\right)\right]$$

K is the *Kalman Gain* matrix. A solution is

$$\mathcal{K}_i = \Sigma_i^- \mathcal{M}_i^T \left[\mathcal{M}_i \Sigma_i^- \mathcal{M}_i^T + \Sigma_{m_i} \right]^{-1}$$

Kalman Gain Matrix

$$\overline{\boldsymbol{x}}_i^+ = \overline{\boldsymbol{x}}_i^- + \mathcal{K}_i \left[\boldsymbol{y}_i - \mathcal{M}_i \overline{\boldsymbol{x}}_i^- \right]$$

$$\mathcal{K}_i = \Sigma_i^- \mathcal{M}_i^T \left[\mathcal{M}_i \Sigma_i^- \mathcal{M}_i^T + \Sigma_{m_i} \right]^{-1}$$

As measurement becomes more reliable, K weights residual more heavily,

$$\lim_{\Sigma_m \to 0} K_i = M^{-1}$$

As prior covariance approaches 0, measurements are ignored:

$$\lim_{\Sigma_i^- \to 0} K_i = 0$$

Embellishments for tracking

Richer models of $P(x_n|x_{n-1})$

Richer models of $P(y_n|x_n)$

Richer model of probability distributions

Abrupt changes

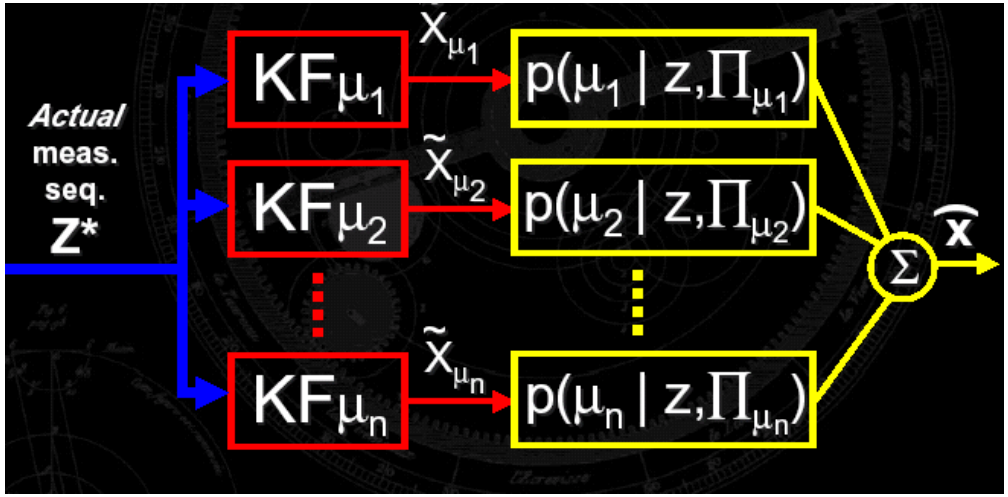
What if environment is sometimes unpredictable?

Do people move with constant velocity?

Test several models of assumed dynamics, use the best.

Multiple model filters

Test several models of assumed dynamics



$$p_j(k) = \frac{f(\vec{z}|\mu_j)p_j(k-1)}{\sum_{h=1}^{r} f(\vec{z}|\mu_h)p_h(k-1)}$$

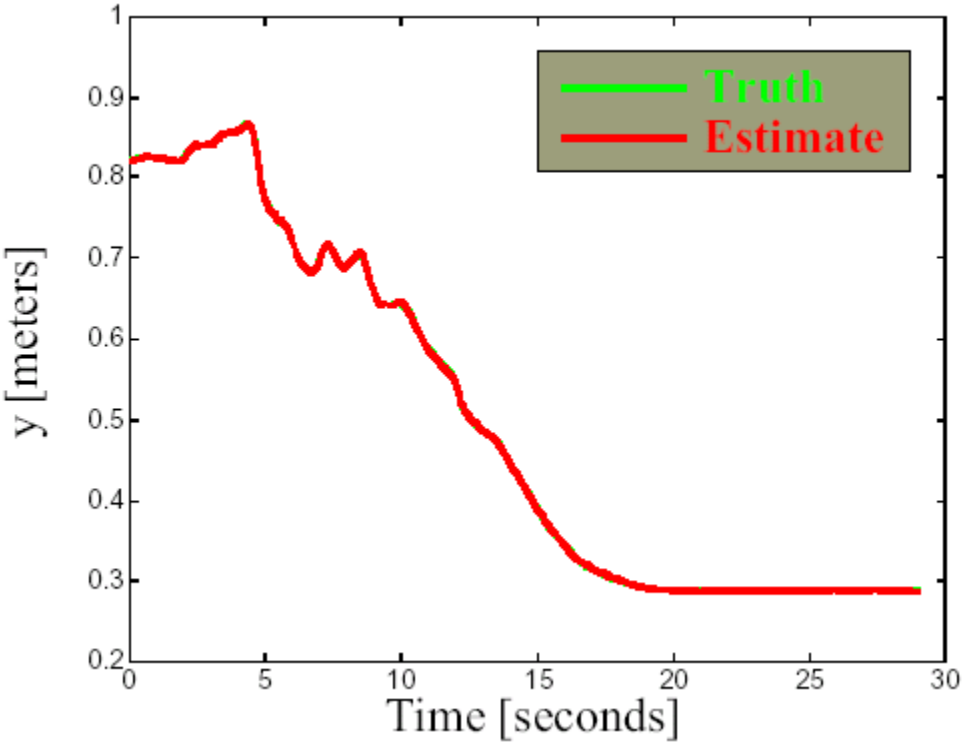
$$p_j(k) = \frac{f(\vec{z}|\mu_j)}{\sum_{h=1}^{r} f(\vec{z}|\mu_h)}$$

$$f(\vec{z}|\mu) = \frac{1}{(2\pi)^{n_\mu/2}\sqrt{|C_\mu|}} e^{-\frac{1}{2}(\vec{z}-H_\mu\vec{x}_\mu)^T C_\mu^{-1}(\vec{z}-H_\mu\vec{x}_\mu)}$$

[figure from Welsh and Bishop 2001]

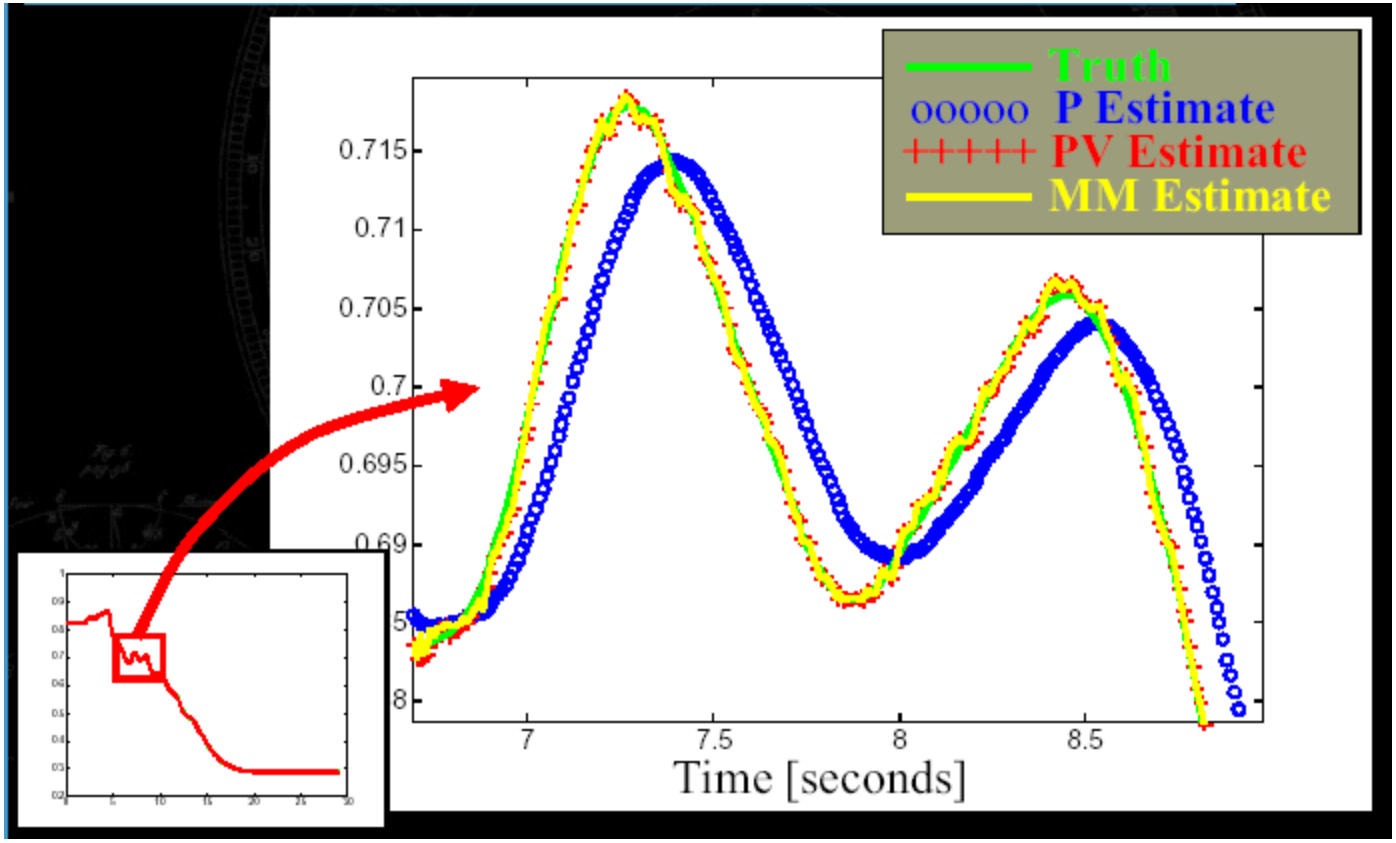
MM estimate

Two models: Position (P), Position+Velocity (PV)



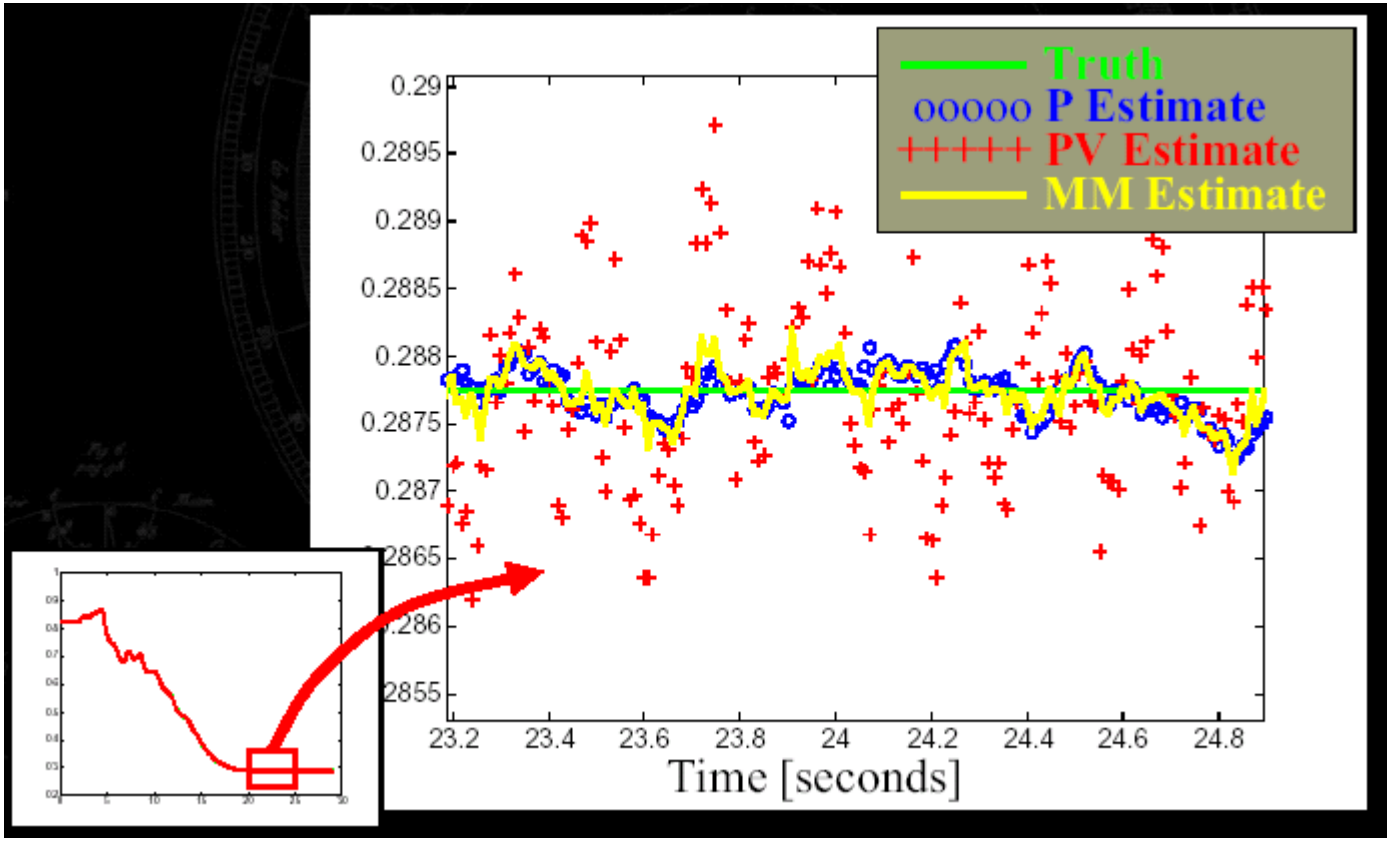
[figure from Welsh and Bishop 2001]

No lag



[figure from Welsh and Bishop 2001]

Smooth when still



[figure from Welsh and Bishop 2001]

Embellishments for tracking

Richer models of $P(y_n|x_n)$

Richer model of probability distributions

Jepson, Fleet, and El-Maraghi tracker

IEEE Conference on Computer Vision and and Pattern Recognition, Kauai, 2001, Vol. I, pp. 415–422

Robust Online Appearance Models for Visual Tracking

Allan D. Jepson* **David J. Fleet†** **Thomas F. El-Maraghi*†**

* Department of Computer Science, University of Toronto, Toronto, M5S 1A4
† Xerox Palo Alto Research Center, 3333 Coyote Hill Rd, Palo Alto, CA 94304

Wandering, Stable, and Lost appearance model

Introduce 3 competing models to explain the appearance of the tracked region:

- A stable model—Gaussian with some mean and covariance.
- A 2-frame motion tracker appearance model, to rebuild the stable model when it gets lost
- An outlier model—uniform probability over all appearances.

Use an on-line EM algorithm to fit the (changing) model parameters to the recent appearance data.

Embellishments for tracking

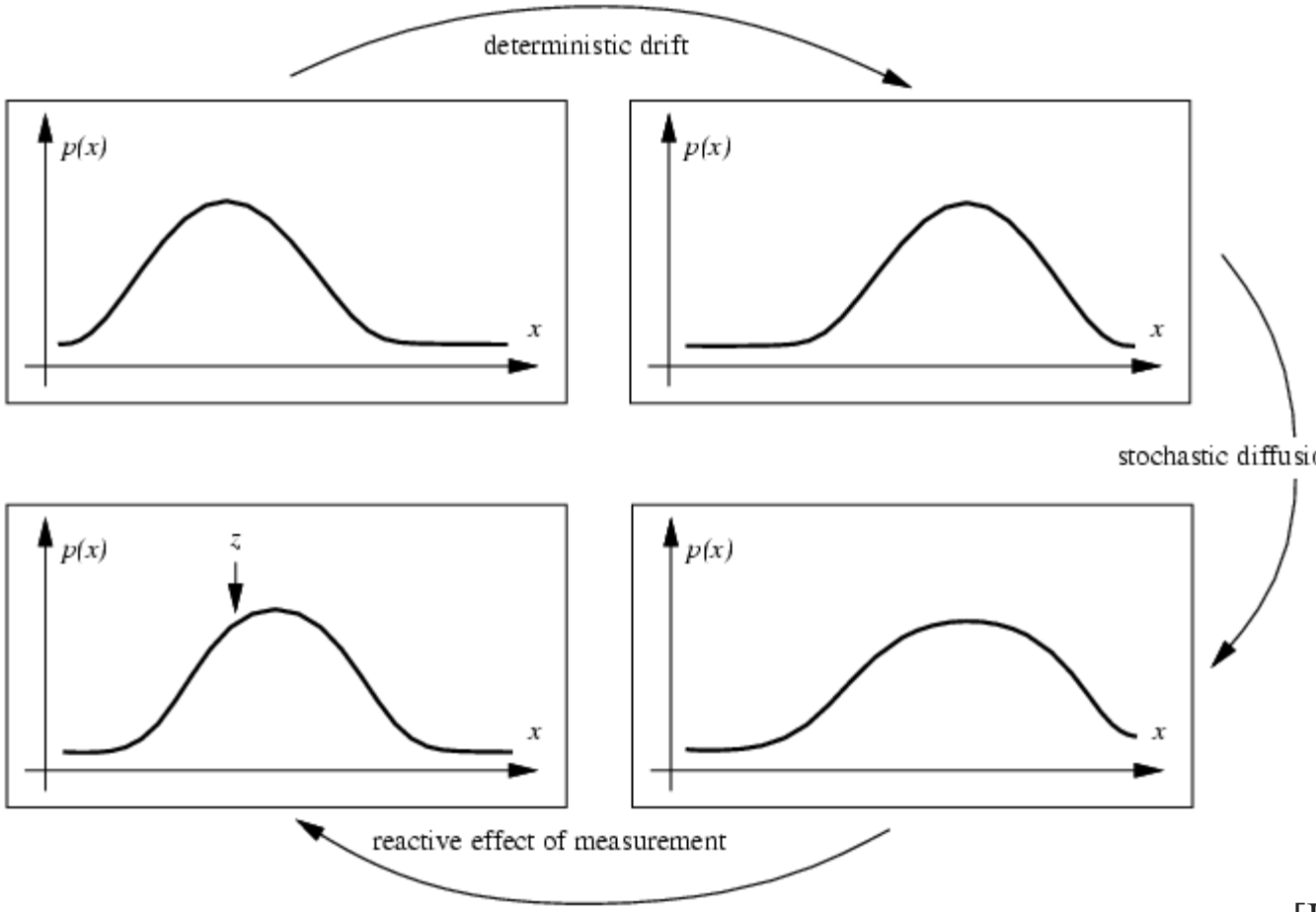
Richer model of $P(y_n|x_n)$

Richer model of probability distributions

Particle filter models, applied to tracking humans

(KF) Distribution propagation

prediction from previous time frame

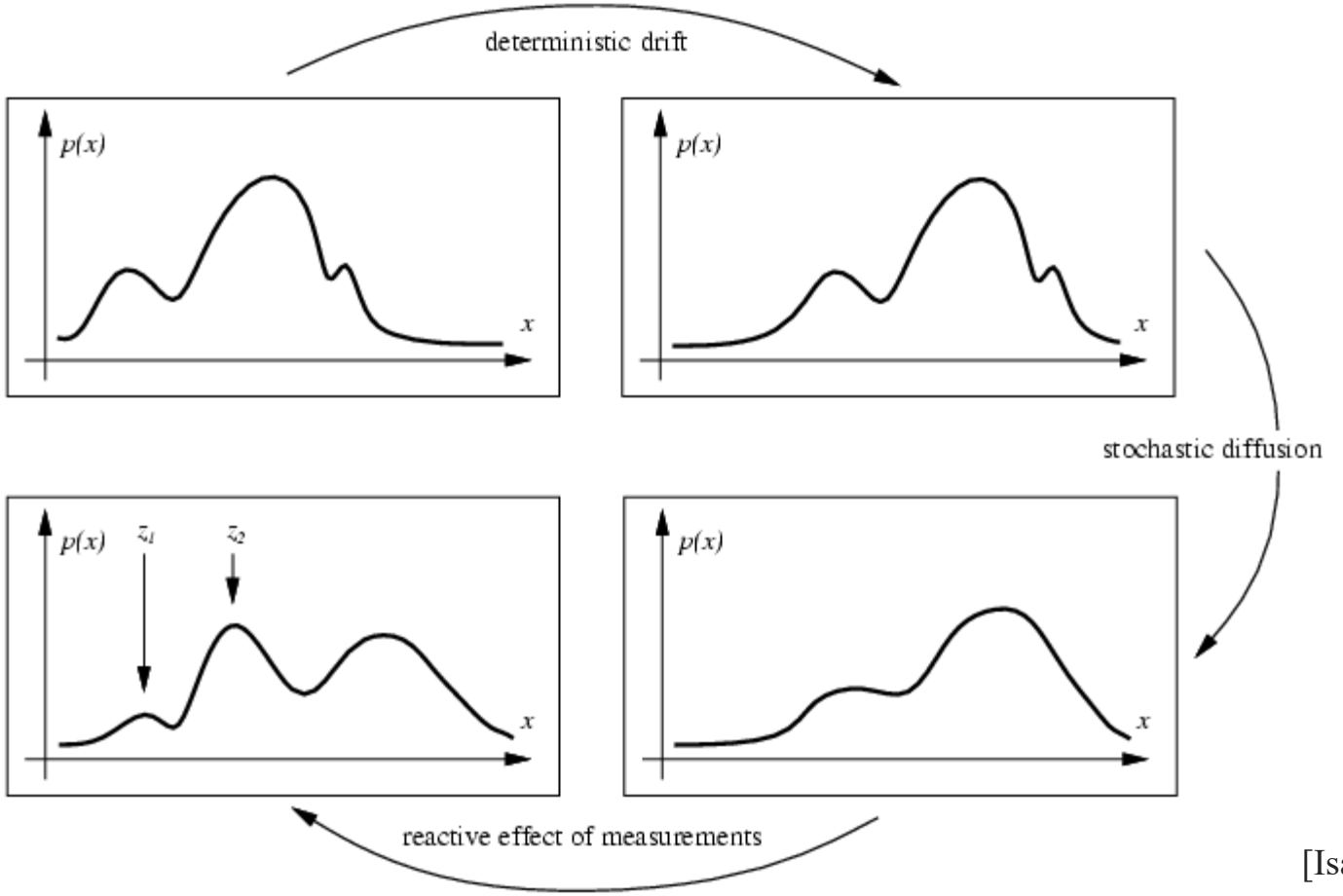


Noise added to that prediction

[Isard 1998]

Make new measurement at next time frame

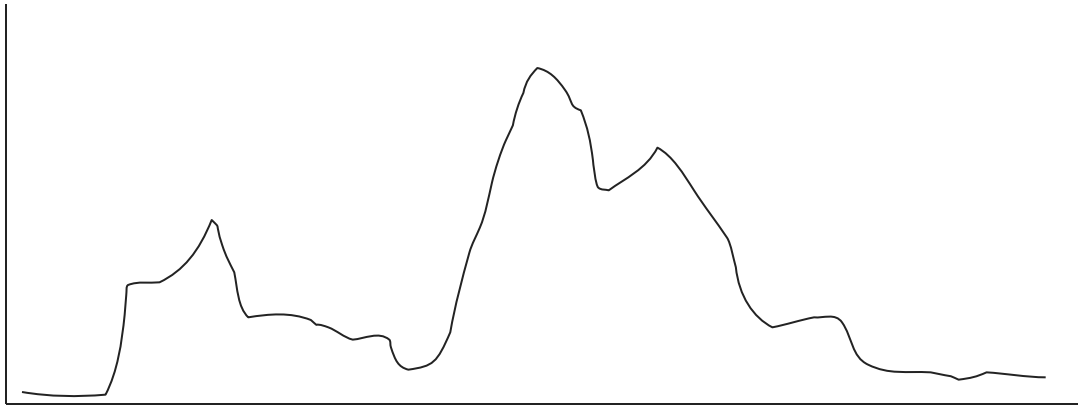
Distribution propagation



[Isard 1998]

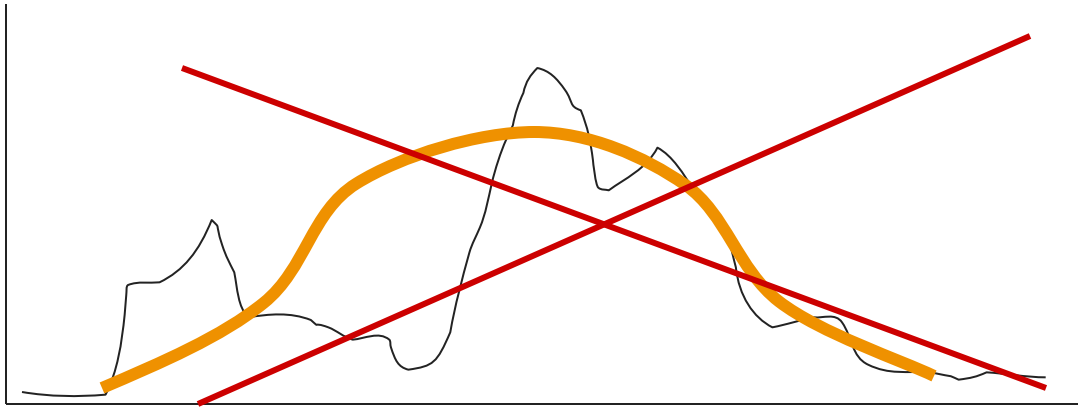
Representing non-linear Distributions

$$p(\mathbf{x}_t|\mathcal{Z}_t) = k_t \; p(\mathbf{z}_t|\mathbf{x}_t)p(\mathbf{x}_t|\mathcal{Z}_{t-1})$$



Representing non-linear Distributions

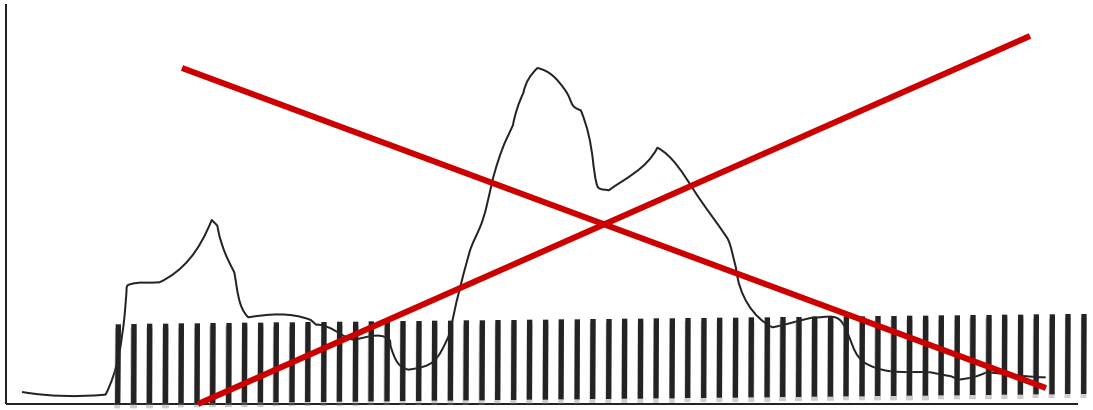
Unimodal parametric models fail to capture real-world densities…



Discretize by evenly sampling over the entire state space

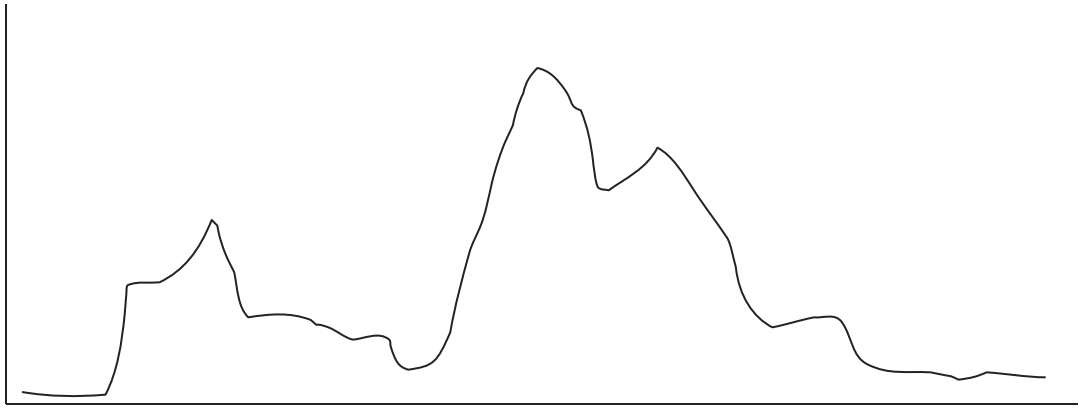
Tractable for 1-d problems like stereo, but not for high-dimensional problems.

$$p(\mathbf{x}|\mathbf{z}) = kp(\mathbf{z}|\mathbf{x})p(\mathbf{x})$$



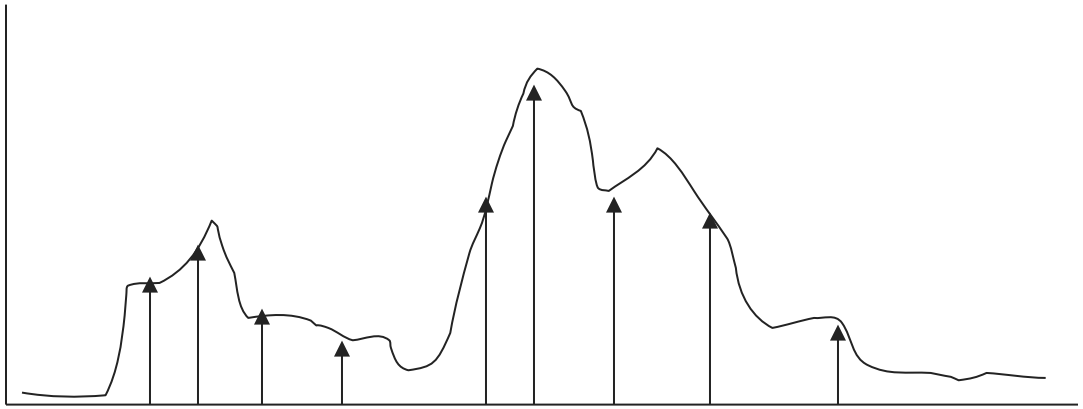
Representing Distributions using Weighted Samples

Rather than a parametric form, use a set of samples to represent a density:



Representing Distributions using Weighted Samples

Rather than a parametric form, use a set of samples to represent a density:



$$\{(\boldsymbol{u}^i, w^i)\}$$

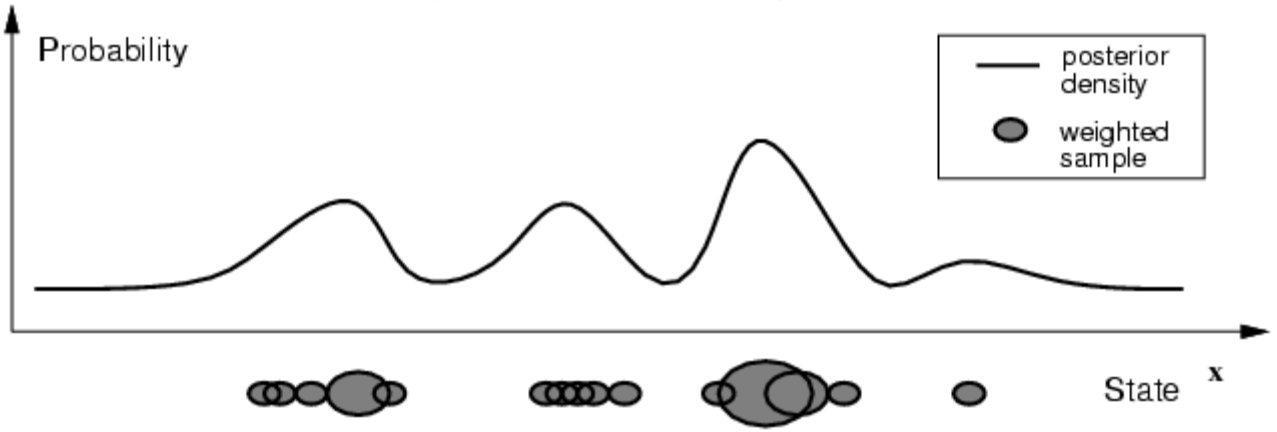
Sample positions

Probability mass at each sample

This gives us two knobs to adjust when representing a probability density by samples: the locations of the samples, and the probability weight on each sample.

Representing distributions using weighted samples, another picture

$$p(\mathbf{x}|\mathbf{z}) = kp(\mathbf{z}|\mathbf{x})p(\mathbf{x})$$



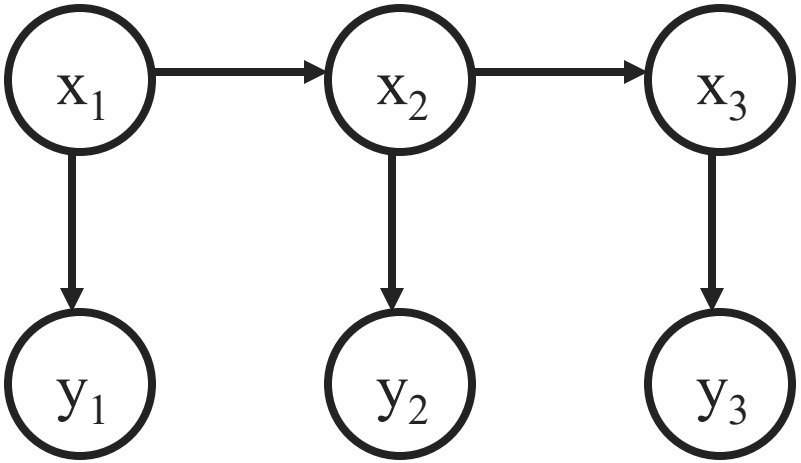
$$\pi_i = \frac{p_z(\mathbf{s}^{(i)})}{\sum_{j=1}^{N} p_z(\mathbf{s}^{(j)})}$$

$$p_f(x) = \sum_i w^i \delta(x - u^i)$$

$$p_z(\mathbf{x}) = p(\mathbf{z}|\mathbf{x}),$$

[Isard 1998]

Tracking, in particle filter representation



$$p_f(x) = \sum_i w^i \delta(x - u^i)$$

$$P(x_n \mid y_1 \ldots y_n) = k\, P(y_n \mid x_n) \int dx_{n-1} P(x_n \mid x_{n-1}) P(x_{n-1} \mid y_1 \ldots y_{n-1})$$

Update step

Prediction step

Particle filter

Let's apply this sampled probability density machinery to generalize the Kalman filtering framework.

More general probability density representation than uni-modal Gaussian.

Allows for general state dynamics, f(x) + noise

Sampled Prediction

$$P(\boldsymbol{x}_i|\boldsymbol{y}_0,\cdots,\boldsymbol{y}_{i-1}) \;=\; ?$$

$$p(\boldsymbol{X}_i,\boldsymbol{X}_{i-1}|\boldsymbol{y}_0,\cdots,\boldsymbol{y}_{i-1}) = p(\boldsymbol{X}_i|\boldsymbol{X}_{i-1})p(\boldsymbol{X}_{i-1}|\boldsymbol{y}_0,\cdots,\boldsymbol{y}_{i-1})$$

$$p(\boldsymbol{X}_{i-1}|\boldsymbol{y}_0,\cdots,\boldsymbol{y}_{i-1})$$

$$\{(\boldsymbol{u}_{i-1}^k, w_{i-1}^k)\} \longrightarrow \boldsymbol{x}_i = \boldsymbol{f}(\boldsymbol{x}_{i-1}) + \xi_i \longrightarrow$$

$$\{((f(\boldsymbol{u}_{i-1}^k) + \xi_i^l, \boldsymbol{u}_{i-1}^k), w_{i-1}^k)\}$$

$$p(\boldsymbol{X}_i,\boldsymbol{X}_{i-1}|\boldsymbol{y}_0,\cdots,\boldsymbol{y}_{i-1})$$

Drop elements to marginalize to get

$$P(\boldsymbol{x}_i|\boldsymbol{y}_0,\cdots,\boldsymbol{y}_{i-1}) \;\sim=$$

$$\{(f(\boldsymbol{u}_{i-1}^k) + \xi_i^l, w_{i-1}^k)\}$$

Sampled Correction (Bayes rule)

Prior → posterior

Reweight every sample with the likelihood of the observations, given that sample:

$$p(\boldsymbol{Y}_i = \boldsymbol{y}_i | \boldsymbol{X}_i = \boldsymbol{s}_i^{k,-}) w_i^{k,-}$$

yielding a set of samples describing the probability distribution after the correction (update) step:

$$\left\{ (\boldsymbol{s}_i^{k,-}, p(\boldsymbol{Y}_i = \boldsymbol{y}_i | \boldsymbol{X}_i = \boldsymbol{s}_i^{k,-}) w_i^{k,-}) \right\}$$

Naïve PF Tracking

Start with samples from something simple (Gaussian)

Repeat

Take each particle from the prediction step and modify the old weight by multiplying by the new likelihood

– Correct

$$\left\{(\boldsymbol{s}_i^{k,-}, p(\boldsymbol{Y}_i = \boldsymbol{y}_i | \boldsymbol{X}_i = \boldsymbol{s}_i^{k,-}) w_i^{k,-})\right\}$$

– Predict

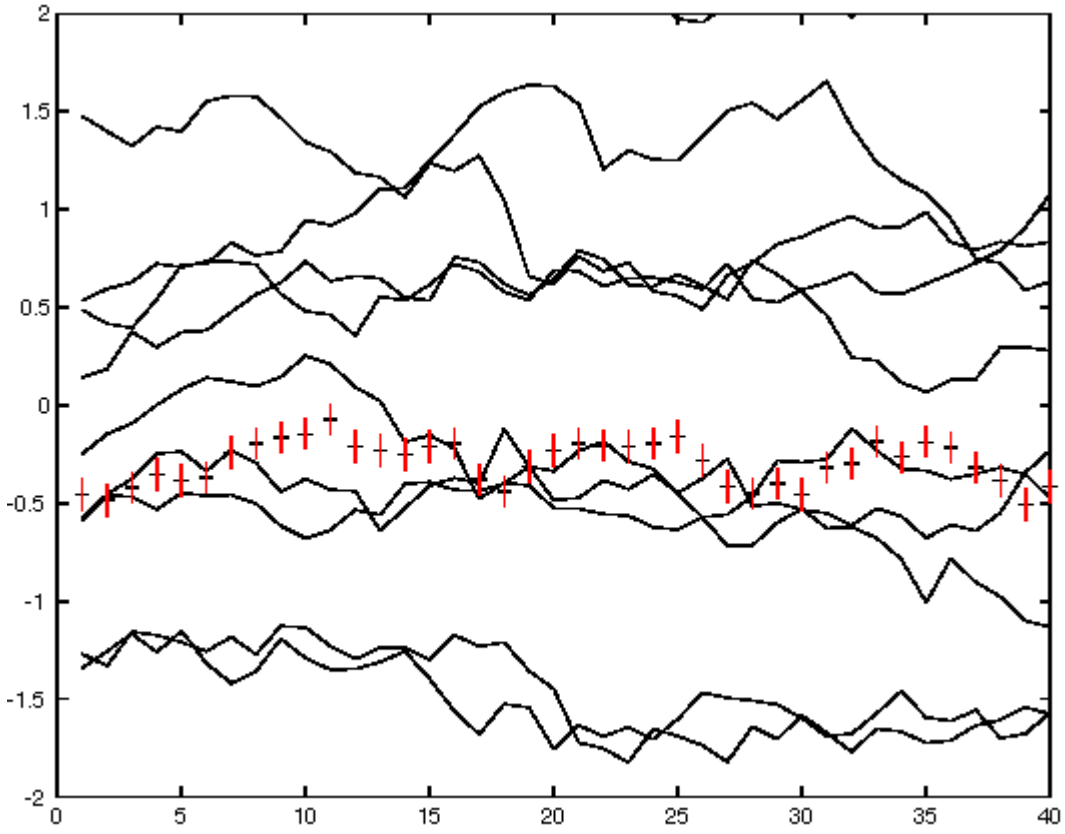
$$\left\{(f(\mathbf{s}_{i-1}^k) + \xi_i^l, w_{i-1}^k)\right\}$$

Run every particle through the dynamics function and add noise.

But doesn't work that well because of sample impoverishment…

Sample impoverishment

10 of the 100 particles, along with the true Kalman filter track, with variance:



Resample the prior

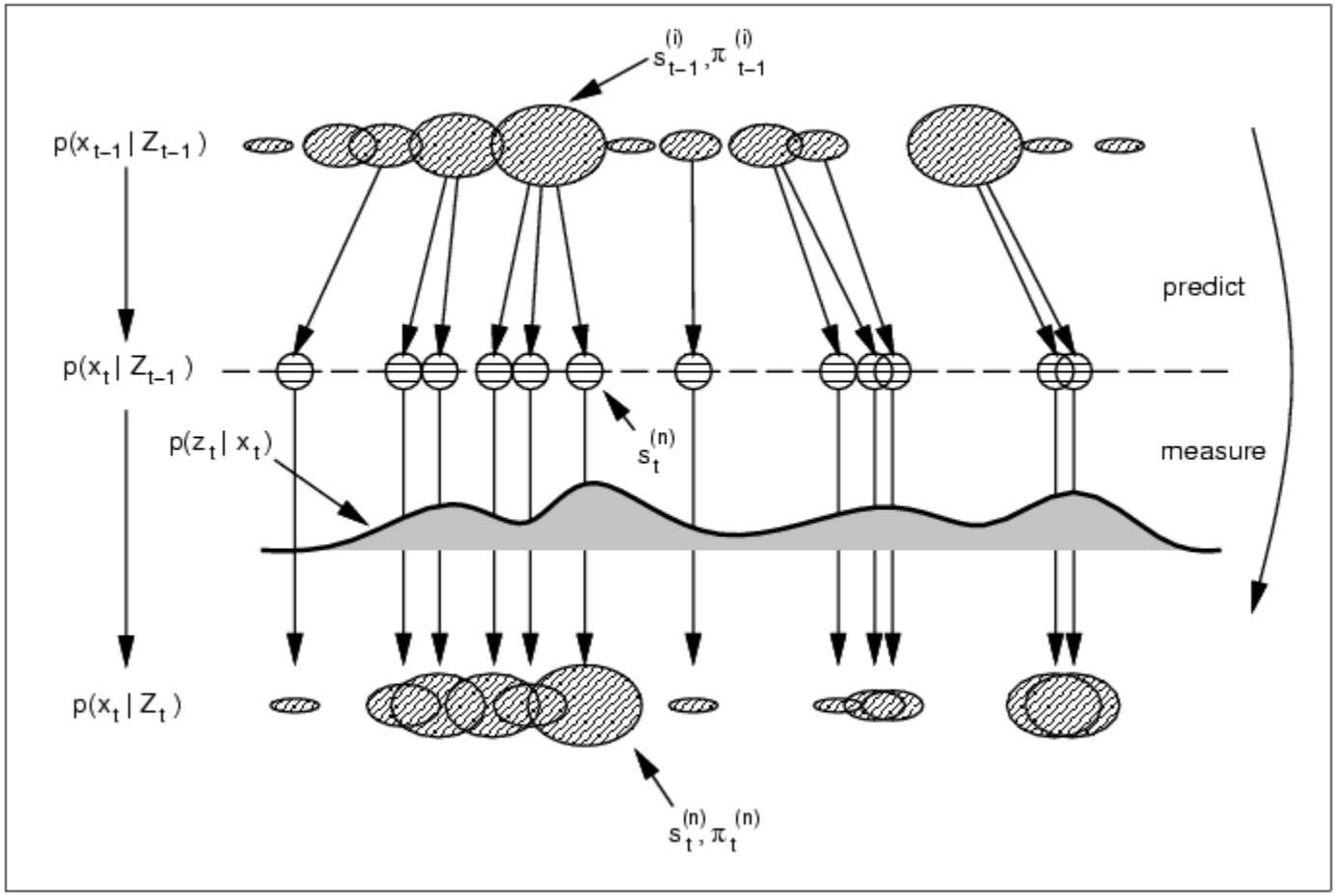
In a sampled density representation, the frequency of samples can be traded off against weight:

$$(\boldsymbol{s}_k, w_k) \longrightarrow \begin{matrix}(\boldsymbol{s}_k, 1)\\ (\boldsymbol{s}_k, 1)\\ (\boldsymbol{s}_k, 1)\\ \vdots\end{matrix} \quad N_k \text{ copies} \quad \text{s.t.} \quad \frac{N_k}{\sum_k N_k} = w_k$$

These new samples are a representation of the same density.

I.e., make N draws with replacement from the original set of samples, using the weights as the probability of drawing a sample.

Pictorial view



[Isard 1998]

```
# 初始化
particles = sample_gauss(init_box, sigma0, N)   # x_0^(i)
weights   = np.ones(N) / N

for t in range(1, T):
    # 1. 预测：状态转移采样（先验）
    particles = motion_model(particles) + np.random.normal(0, sigma_dyn, particles.shape)

    # 2. 权重更新：外观似然
    for i in range(N):
        patch = crop(frame[t], particles[i])
        d = bhattacharyya(hist_model, hist(patch))     # 距离 [0,1]
        weights[i] = np.exp(-lam * d**2)               # 似然

    weights_sum = weights.sum()
    if weights_sum < 1e-12:          # 全失效 → 重新初始化
        particles = sample_gauss(prev_box, sigma0, N);
        weights = np.ones(N)/N
    else:       weights /= weights_sum

    # 3. 状态估计（加权均值）
    x_hat = (weights[:, None] * particles).sum(axis=0)
    output(x_hat)
```

```
    # 4. 自适应重采样
    n_eff = 1.0 / (weights**2).sum()
    if n_eff < N / 2:
        idx = systematic_resample(weights)
        particles = particles[idx]
        weights = np.ones(N) / N

    # 5. t += 1，回到 1
```

Animation of condensation algorithm



[Isard 1998]

Applications

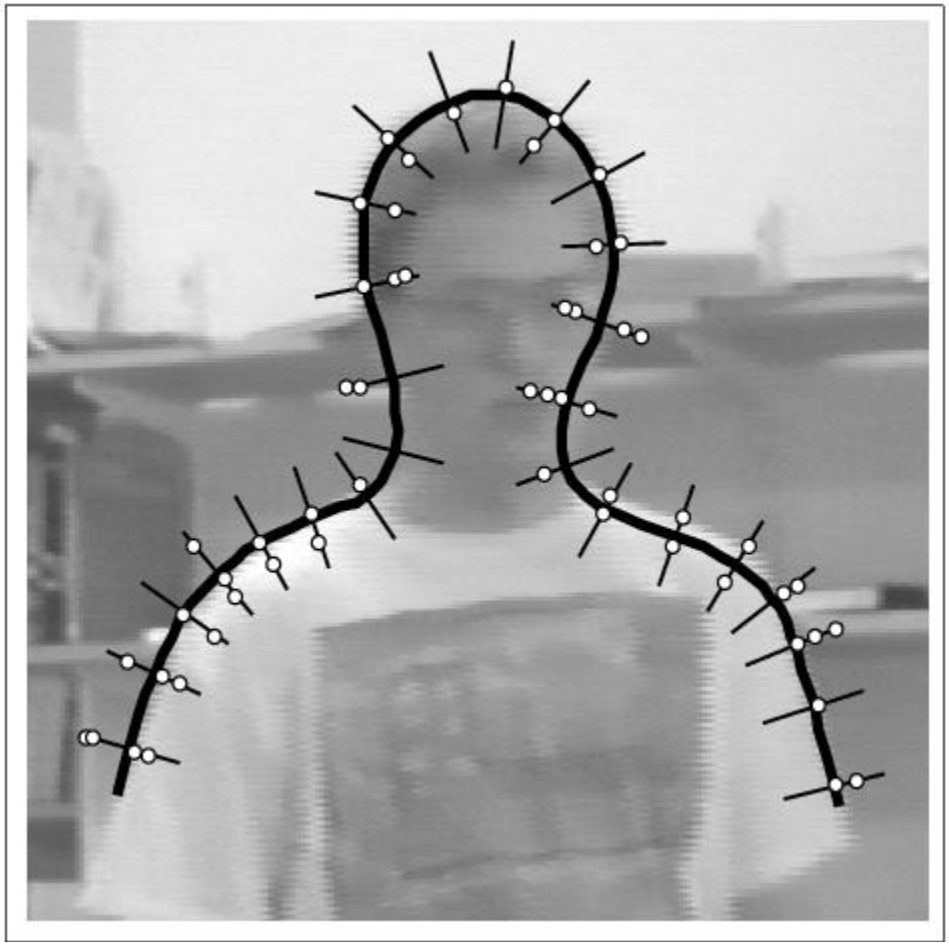
Tracking

- hands
- bodies
- Leaves

What might we expect?

Reliable, robust, slow

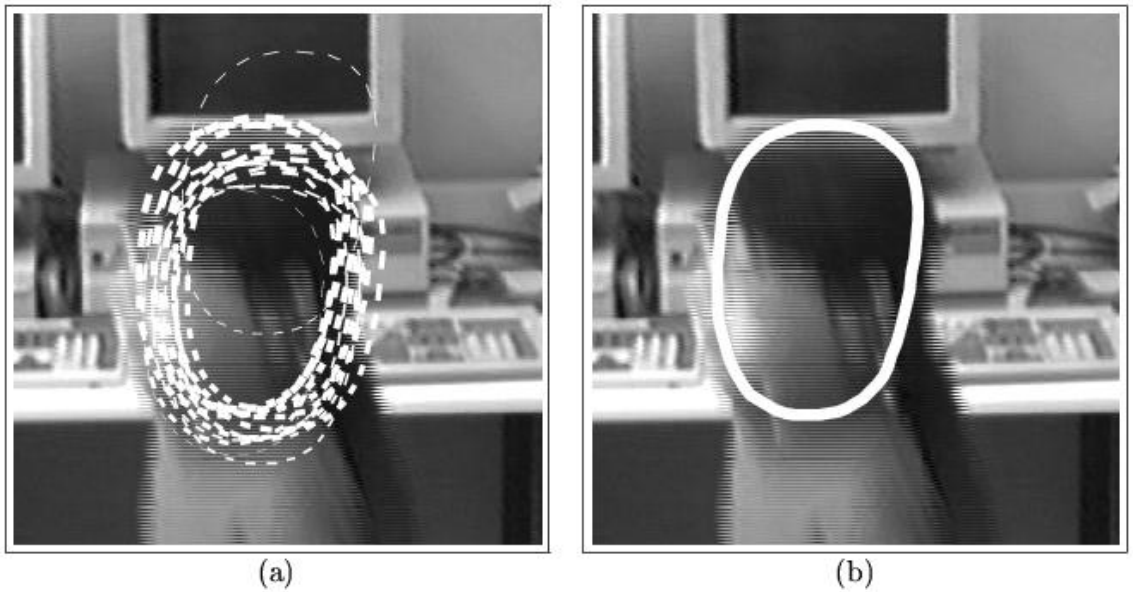
Contour tracking



[Isard 1998]

Head tracking





(a) (b)

The mean state

Picture of the states represented by the top weighted particles

[Isard 1998]

Leaf tracking



[Isard 1998]

Hand tracking



[Isard 1998]